MacBASIC

A Retro BASIC Interpreter for the Browser

USER MANUAL & LANGUAGE REFERENCE

v1.0

Line numbers. GOTO. A blinking cursor. Everything you remember — running entirely in one HTML file.

1. What Is MacBASIC?

MacBASIC is a self-contained line-numbered BASIC interpreter that runs entirely inside a single HTML file — no installation, no server, no build step. Open the file in any modern browser and you get a green-phosphor CRT terminal, complete with scanlines, screen flicker, and a blinking block cursor, running a real interpreter underneath: line numbers, GOTO, GOSUB/RETURN, FOR/NEXT, arrays, string functions, and a small set of graphics commands (PSET, LINE, CIRCLE) for drawing simple scenes and building games.

It is meant for exactly the kind of tinkering BASIC was originally built for: type a line, run it, see what happens, change it, run it again. There is no project setup and no compile step between you and a working program.

1.1 What You Can Build

- Simple utilities and calculators using loops, arrays, and string manipulation.
- Text adventures and choice-driven games using GOSUB-based rooms and inventory flags.
- Drawn scenes and simple animations using the graphics commands and SLEEP.
- Live-updating dashboards (like a simulated system monitor) that redraw on a loop.

1.2 How the Screen Works

The terminal has two layers sharing the same space: a scrolling text layer (everything PRINT and INPUT produce) and a transparent graphics canvas on top of it (everything PSET, LINE, and CIRCLE draw). Because the canvas is transparent, text and graphics can appear together on screen at the same time.

2. Getting Started

2.1 Typing a Program

Every line that begins with a number is stored as part of your program, in numeric order, regardless of the order you typed them in. A line typed without a leading number runs immediately instead (this is called **immediate mode**) — useful for quick one-off calculations or checking a variable's value.

Example:

```
10 PRINT "HELLO, WORLD!"
20 FOR I = 1 TO 3
30 PRINT "COUNT: "; I
40 NEXT I
RUN
```

2.2 Editing and Removing Lines

Type a line number that already exists to replace that line. Type a line number by itself (nothing after it) to delete that line entirely.

2.3 Core Session Commands

Command	What it does
`RUN`	Runs the stored program from the lowest line number.
`LIST`	Prints every stored line, in order.
`NEW`	Clears the current program and all variables.
`CLS / CLEAR`	Clears the screen (text and any drawn graphics).
`HELP`	Shows an in-terminal quick reference.

2.4 Interacting With the Terminal

- **Up / Down arrows** recall previous lines you typed (command mode only).
- **Ctrl+C** breaks a running program, or cancels a pending INPUT.
- **Click-drag** to select on-screen text, then **Cmd/Ctrl+C** to copy it out normally.
- **Pasting** multiple lines runs each one in turn, as if typed and entered by hand.
- **Dragging a file** onto the screen loads it the same way (see Section 6).

3. Statement Reference

Every statement below can appear on its own line, or be chained with other statements on the same line using a colon (`:`). Keywords are case-insensitive, but this manual writes them in capitals for clarity, as is traditional.

3.1 Output & Assignment

Command	What it does
`PRINT expr, expr, ...`	Prints one or more values. A comma pads to the next 14-column tab stop; a semicolon prints with no gap. A trailing comma or semicolon suppresses the newline.
`LET var = expr`	Assigns a value to a variable. The word LET is optional — `X = 5` works on its own.
`INPUT ["prompt";] var, ...`	Pauses and waits for the person at the keyboard to type a line, then assigns the comma-separated values typed to the listed variables.

3.2 Variables & Types

A variable name ending in $ holds text (a string), e.g. `NAME$`. Any other name holds a number, e.g. `SCORE`. Numeric variables default to 0 and string variables default to an empty string until assigned.

3.3 Control Flow

Command	What it does
`IF cond THEN stmt [ELSE stmt]`	Runs the THEN statement(s) if the condition is true, otherwise the ELSE statement(s) if present. THEN/ELSE may be followed by a bare line number as a shortcut for GOTO, or by one or more statements chained with colons.
`FOR var = start TO end [STEP n]`	Begins a counted loop. STEP defaults to 1 and may be negative to count down.
`NEXT [var]`	Closes the most recent FOR loop, incrementing its counter and looping back if it has not yet passed the end value.
`GOTO line`	Jumps directly to the given line number.
`GOSUB line`	Jumps to a subroutine at the given line number, remembering where to return to.
`RETURN`	Returns from a subroutine to the statement right after the GOSUB that called it.
`END / STOP`	Stops the program immediately.

Note: a GOSUB/RETURN pair does not have to sit on the same line, and subroutines can call other subroutines — a common pattern is one GOSUB per "room" or game state, dispatched from a main loop with a chain of IF statements.

3.4 Data & Arrays

Command	What it does
`DIM name(size)`	Reserves an array with indices 0 through size. Use a trailing $ for an array of strings, e.g. `DIM NAMES$(10)`.
`DATA val, val, ...`	Declares a list of constant values (numbers or quoted strings), read in order by READ statements anywhere in the program.
`READ var, var, ...`	Reads the next value(s) from the program's DATA list into variables.
`RESTORE`	Resets the DATA pointer back to the first value, so it can be READ again from the start.

3.5 Miscellaneous

Command	What it does
`REM comment`	A remark/comment. Everything after REM on that line is ignored (including any colons).
`CLS`	Clears both the text screen and any drawn graphics.
`SLEEP n`	Pauses execution for n seconds (can be fractional, e.g. `SLEEP 0.5`). Useful for pacing animations or live-updating displays.

4. Function Reference

4.1 Math

Command	What it does
`ABS(x)`	Absolute value.
`INT(x)`	Rounds down to the nearest integer.
`SGN(x)`	Sign of x: -1, 0, or 1.
`SQR(x)`	Square root.
`RND(x)`	A random number between 0 and 1 (the argument is accepted but ignored, for compatibility).
`SIN(x) / COS(x) / ATN(x)`	Trigonometric functions, in radians.
`LOG(x) / EXP(x)`	Natural logarithm and e raised to the power x.

4.2 Strings

Command	What it does
`LEN(s$)`	Length of a string.
`LEFT$(s$, n) / RIGHT$(s$, n)`	The leftmost or rightmost n characters of a string.
`MID$(s$, start, len)`	A substring starting at position start (1-based) for len characters.
`STR$(n)`	Converts a number to its string representation.
`VAL(s$)`	Converts a string to a number (0 if it isn't numeric).
`CHR$(n)`	The character for a given character code.
`ASC(s$)`	The character code of a string's first character.

4.3 Formatting

Command	What it does
`TAB(n)`	Used inside a PRINT statement to pad output out to column n.

5. Graphics

MacBASIC draws onto a 320×200 logical canvas that sits transparently on top of the text screen, then stretches to fill the visible terminal with chunky, pixelated scaling to keep the low-resolution retro look.

Command	What it does
`SCREEN 1`	Clears the graphics layer, ready for drawing.
`PSET (x, y) [, c]`	Plots a single pixel at (x, y) in color c (0–15). Color defaults to the current COLOR if omitted.
`LINE (x1,y1)-(x2,y2) [, c[, B|BF]]`	Draws a line between two points. Add B for a box outline between the two points as opposite corners, or BF for a filled box.
`CIRCLE (x, y), r [, c]`	Draws a circle outline of radius r centered at (x, y).
`COLOR c`	Sets the default pen color (0–15) used when PSET/LINE/CIRCLE omit one.

5.1 Color Palette

Colors are numbered 0–15, a classic 16-color palette:

#	Swatch	Name	#	Swatch	Name
0		Black	8		Dark Gray
1		Blue	9		Light Blue
2		Green	10		Light Green
3		Cyan	11		Light Cyan
4		Red	12		Light Red
5		Magenta	13		Light Magenta
6		Brown	14		Yellow
7		Light Gray	15		White

5.2 Example

```
10 SCREEN 1
20 COLOR 14
30 CIRCLE (160,100), 40
40 COLOR 2
50 LINE (0,150)-(319,199), 2, BF
```

Draws a yellow circle over a green ground — a sun over grass.

6. Saving, Loading & Sharing Programs

6.1 Built-In Catalog

MacBASIC can remember programs between sessions using a small built-in catalog.

Command	What it does
`SAVE "name"`	Stores the current program under that name.
`LOAD "name"`	Replaces the current program with a previously saved one.
`CATALOG`	Lists every program you have saved.
`KILL "name"`	Deletes a saved program.

When MacBASIC is opened as a Claude artifact, the catalog is stored in Claude's artifact storage and is private to your account. If the HTML file is downloaded and opened directly in a browser instead, it falls back to that browser's own local storage, so the catalog then lives on that one computer, in that one browser, until its site data is cleared.

6.2 Loading a File Directly

You can also drag a plain text file containing a BASIC program straight onto the terminal screen. It loads line by line, exactly as if you had pasted or typed it in yourself. This only works while at the READY. prompt, not mid-program.

6.3 The "Floppy Disk": Exporting Your Whole Catalog

To share more than one program at a time — your entire saved catalog — MacBASIC can bundle everything into a single file, in the spirit of handing someone a floppy disk.

Command	What it does
`EXPORT ["name"]`	Bundles every saved program into one downloadable file (named `floppy.img` by default, or `name.img` if given).

To share it: send both the MacBASIC HTML file and the exported `.img` file to a friend. When they drag your `.img` file onto their own copy of MacBASIC, it detects that it's a whole catalog (rather than a single program) and imports every program from it at once — ready to browse with CATALOG and open with LOAD.

7. Example Programs

7.1 Counting Loop

```
10 FOR I = 1 TO 10
20 IF I / 2 = INT(I / 2) THEN PRINT I; "EVEN" ELSE PRINT I; "ODD"
30 NEXT I
```

7.2 A Subroutine

```
10 PRINT "START"
20 GOSUB 100
30 PRINT "END"
40 END
100 PRINT "IN THE SUBROUTINE"
110 RETURN
```

7.3 Arrays and DATA

```
10 FOR I = 1 TO 3
20 READ N$, A
30 PRINT N$; " IS "; A
40 NEXT I
50 DATA "ALICE", 30, "BOB", 25, "CARL", 40
```

7.4 A Live-Updating Display

```
10 FOR N = 1 TO 20
20 CLS
30 PRINT "TICK "; N
40 SLEEP 0.5
50 NEXT N
```

This pattern — clear the screen, redraw, pause, repeat — is how MacBASIC's included system-monitor demo builds its live gauges and scrolling history graph.

8. Quick Reference Card

Group	Covers
`PRINT / LET / INPUT`	Output, assignment, and keyboard input.
`IF ... THEN ... ELSE`	Conditional branching.
`FOR ... TO ... STEP ... NEXT`	Counted loops.
`GOTO / GOSUB / RETURN`	Jumps and subroutines.
`DIM / DATA / READ / RESTORE`	Arrays and constant data.
`REM / END / STOP / CLS`	Comments, program end, screen clear.
`SLEEP n`	Pause n seconds.
`SCREEN / PSET / LINE / CIRCLE / COLOR`	Graphics.
`RUN / LIST / NEW / HELP`	Session commands.
`SAVE / LOAD / CATALOG / KILL`	Program catalog.
`EXPORT`	Bundle the whole catalog as one shareable file.